



**RV College of
Engineering®**

Go, change the world

Concurrent Servers



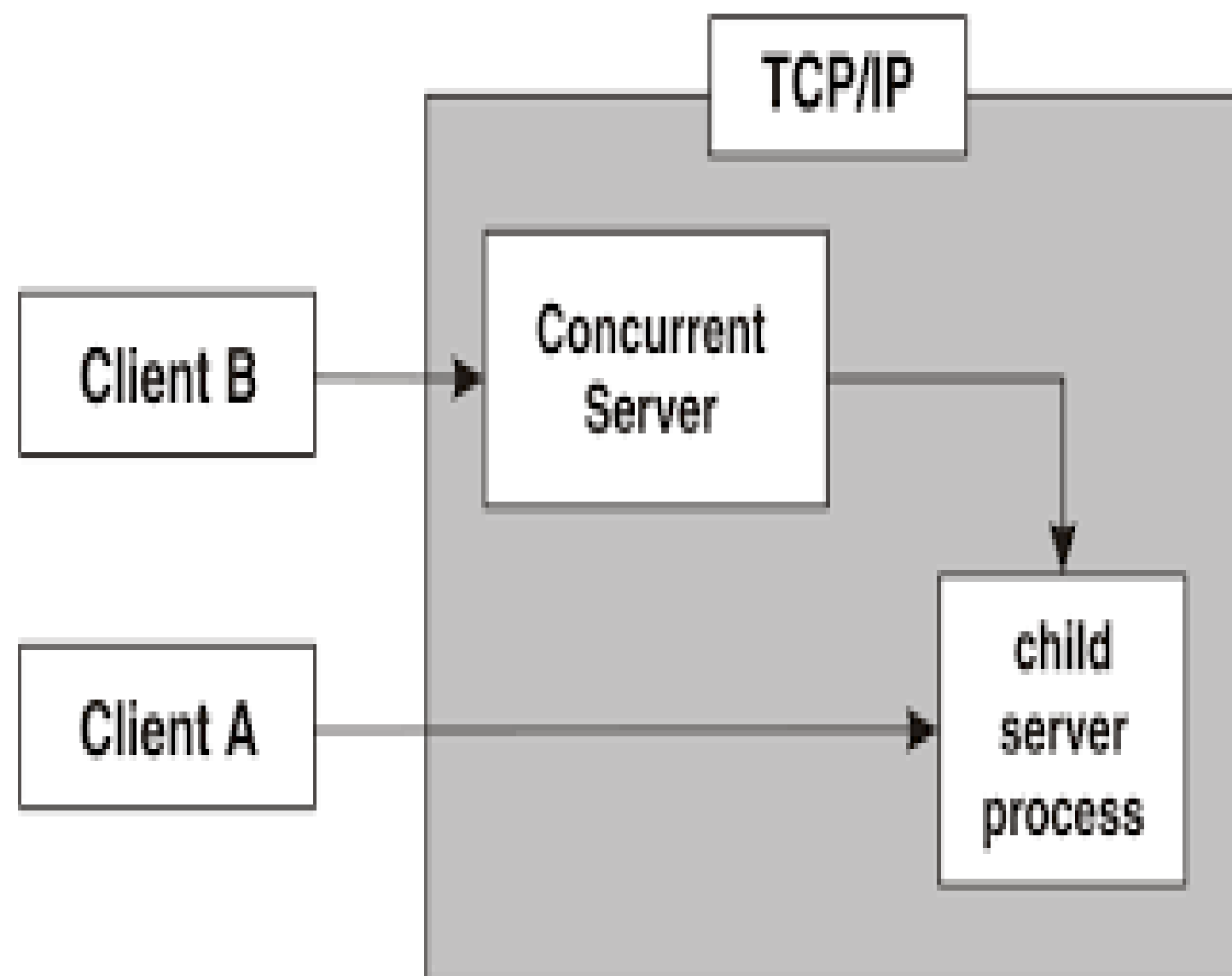
UNIT 2

TCP client/server

Contents

1. Socket function
2. connect function, bind, listen, accept, fork, exec functions
3. Concurrent Server
4. close function
5. getsockname and getpeername functions
6. TCP Echo server – main – str_echo
7. TCP Echo client – main – str_echo
8. Normal startup, normal termination

Concurrent Server: The **server** can be iterative, i.e. it iterates through each client and serves one request at a time. Alternatively, a **server** can handle multiple clients at the same time in parallel, and this type of a **server** is called a **concurrent server**.



How many concurrent connections can a server handle?
65535 simultaneous connections.

On the TCP level the tuple (source ip, source port, destination ip, destination port) must be unique for each simultaneous connection. That means a single client cannot open more than **65535 simultaneous connections** to a server. But a server can (theoretically) serve **65535 Simultaneous connections** per client.

There are several ways we can implement this server: The simplest technique is to call the Unix **fork()** function. Other techniques are to use threads or to pre-fork a fixed number of children when the function starts.

When the server *receives and accepts the client's connection*, it forks a copy of itself and lets the child handle the client as shown in the figure below:

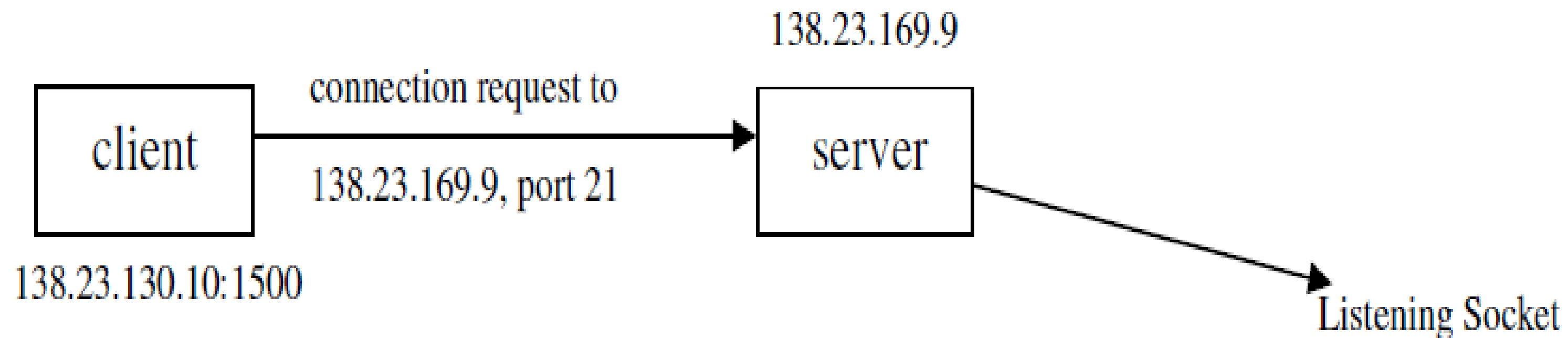


Figure 1: Client with IP address 138.23.130.10 requests to connect to Server (with IP 138.23.169.9 listening from port 21) from its local port number 1500.

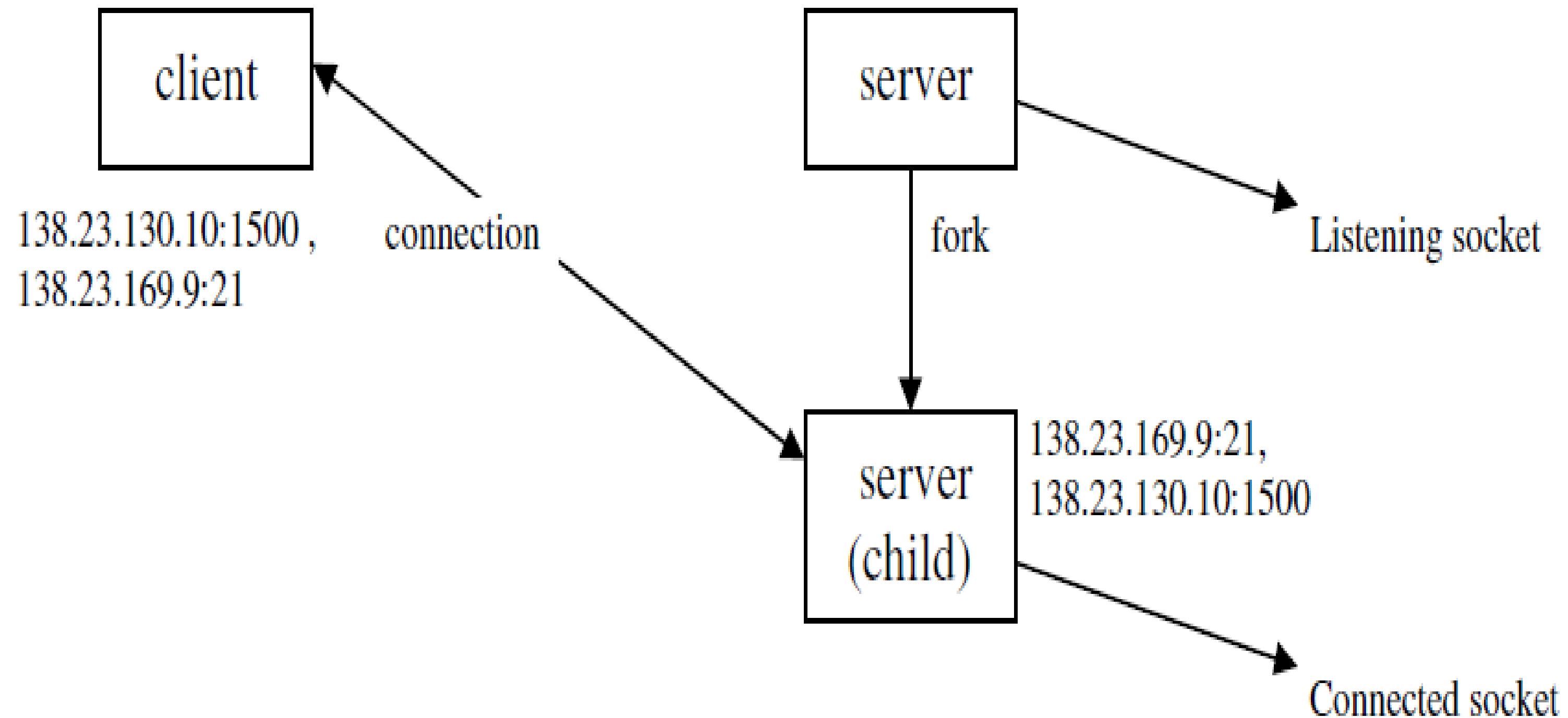


Figure 2: Server `fork()`s a copy of itself and lets the child handle this client from port 21, but it concurrently keeps listening (by the parent process) from port 21 as well.

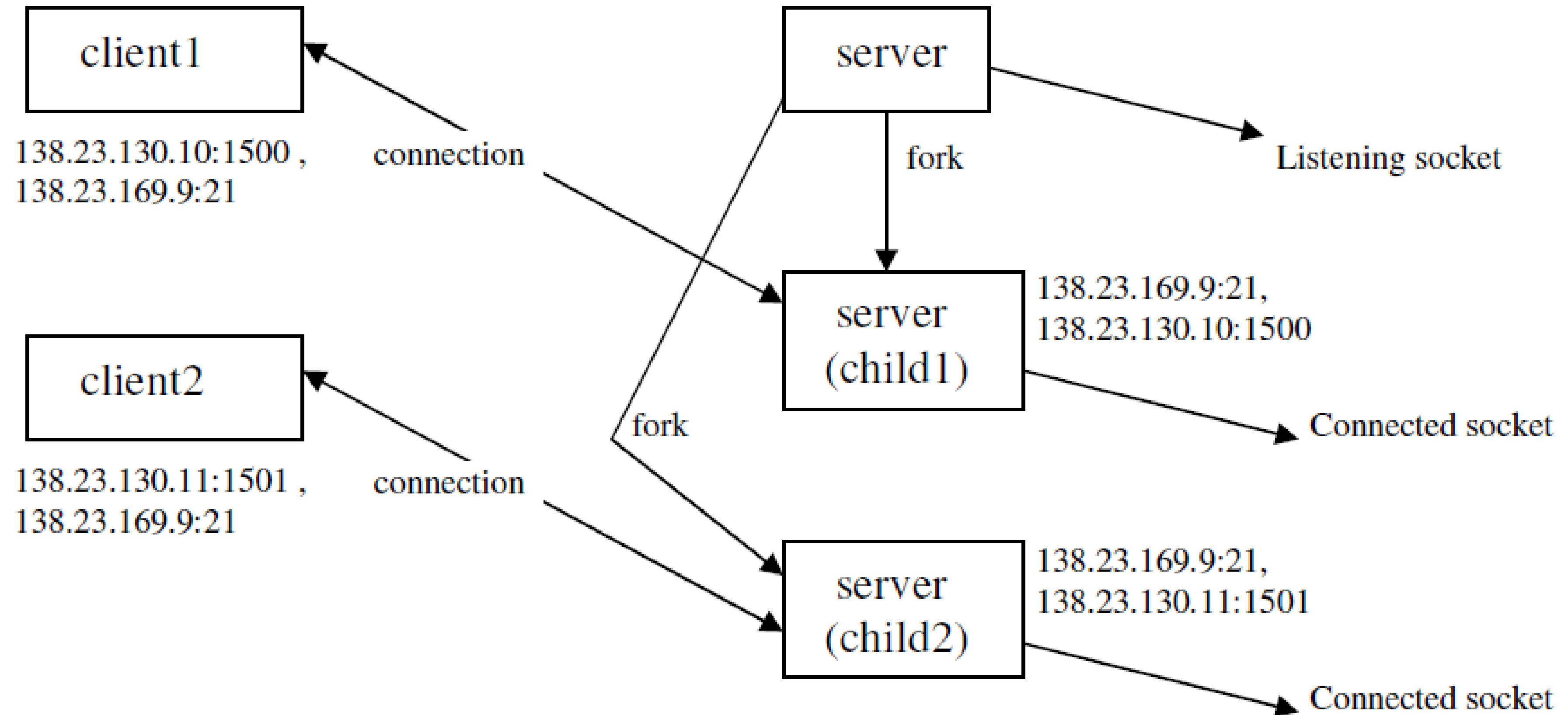


Figure 3: Another client requests to connect to this server. The server fork()s another copy of itself and lets this second child handle this client, whereas it still concurrently keeps listening from port 21 (by the parent process) and keeps serving client1 from port1 (by the first child process).

The listening socket must be distinguished from the connected socket on the server host. Although both sockets use the same local port on the server machine, they are indicated by distinct socket file descriptors, returned server's call of functions `socket()` and `accept()` respectively for listening and connected sockets.

Notice from figure 3 above that TCP must look at all four segments in the socket pair to determine which endpoint receives an arriving segment.

- ❖ In this figure, there are 3 sockets with the same local port 21 on the server.
- ❖ If a segment arrives from 138.23.130.10 port 1500 destined for 138.23.169.9 port 21, it is delivered to the first child.
- ❖ If a segment arrives from 138.23.130.11 port 1501 destined for 138.23.169.9 port 21, it is delivered to the second child.
- ❖ All other TCP segments destined for port 21 are delivered to the original listening socket.

The server described in [intro/daytimetcpsrv1.c](#) is an **iterative server**.

But when a client request can take longer to service, we do not want to tie up a single server with one client; we want to handle multiple clients at the same time.

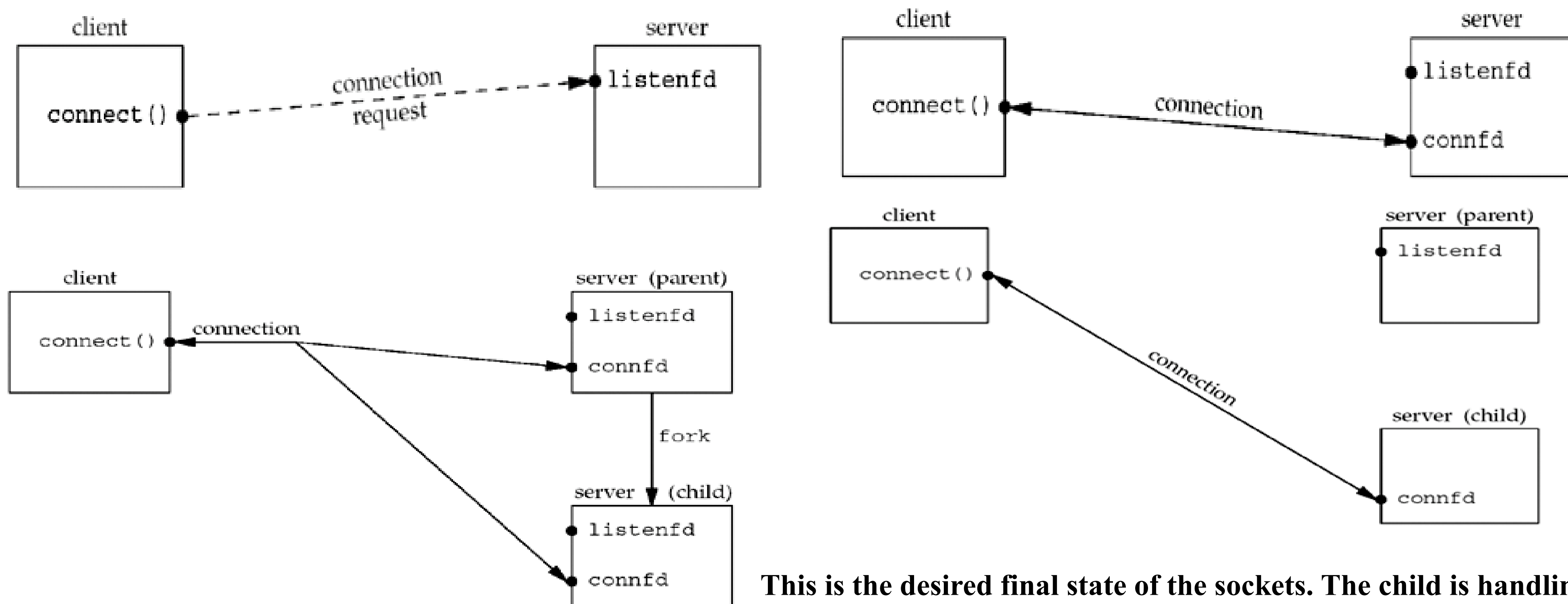
The simplest way to write a concurrent server under Unix is to fork a child process to handle each client.

The following code shows the outline for a typical concurrent server:

```
pid_t pid;
int listenfd, connfd;
listenfd = Socket( ... ); /* fill in sockaddr_in{} with server's well-known port */
Bind(listenfd, ... );
Listen(listenfd, LISTENQ);
for ( ; ; )
{ connfd = Accept (listenfd, ... ); /* probably blocks */
if( (pid = Fork()) == 0)
{ Close(listenfd); /* child closes listening socket */
doit(connfd); /* process the request */
Close(connfd); /* done with this client */
exit(0); /* child terminates */
}
Close(connfd); /* parent closes connected socket */
}
```

- ❖ When a connection is established, accept returns, the server calls fork, and the child process services the client (on connfd, the connected socket) and the parent process waits for another connection (on listenfd, the listening socket).
- ❖ The parent closes the connected socket since the child handles the new client.
- ❖ We assume that the function doit does whatever is required to service the client.
- ❖ When this function returns, we explicitly close the connected socket in the child.
- ❖ This is not required since the next statement calls exit, and part of process termination is to close all open descriptors by the kernel.
- ❖ Whether to include this explicit call to close or not is a matter of personal programming taste.

The following figures visualize the sockets and connection in the code above: Before call to accept returns, the server is blocked in the call to accept and the connection request arrives from the client:



This is the desired final state of the sockets. The child is handling the connection with the client and the parent can call accept again on the listening socket, to handle the next client connection.

The normal Unix close function is also used to close a socket and terminate a TCP connection.

```
#include <unistd.h>
int close (int sockfd);
/* Returns: 0 if OK, -1 on error */
```

The default action of close with a TCP socket is to mark the socket as closed and return to the process immediately.

The socket descriptor is no longer usable by the process: It cannot be used as an argument to read or write.

But, TCP will try to send any data that is already queued to be sent to the other end, and after this occurs, the normal TCP connection termination sequence takes place.

SO_LINGER socket option (detailed in [Section 7.5](#)) lets us change this default action with a TCP socket.

- ❖ getsockname returns the local protocol address associated with a socket.
- ❖ getpeername returns the foreign protocol address associated with a socket.

```
#include <sys/socket.h>
```

```
int getsockname(int sockfd, struct sockaddr *localaddr, socklen_t *addrlen);
```

```
int getpeername(int sockfd, struct sockaddr *peeraddr, socklen_t *addrlen);
```

```
/* Both return: 0 if OK, -1 on error */
```

Client without <code>bind()</code>	<code>getsockname()</code>	✔ Client	To know assigned IP/port	Kernel assigns ephemeral port + local IP during <code>connect()</code> based on routing table; these values are not visible to user unless queried explicitly
<code>bind(port=0)</code>	<code>getsockname()</code>	✔ Both	To know chosen port	When port = 0, kernel performs dynamic port allocation from ephemeral range; application must query kernel to retrieve the selected port stored in socket structure
Address family check	<code>getsockname()</code>	✔ Both	General info	Socket internally stores protocol family (AF_INET / AF_INET6) ; needed in generic or reusable code where socket type is not known beforehand
Wildcard server (<code>0.0.0.0</code>)	<code>getsockname()</code>	✔ Server	To know actual interface	<code>INADDR_ANY</code> binds to all interfaces, but on connection kernel resolves a specific local IP based on incoming interface ; only connected socket contains this resolved address
After <code>accept()</code>	<code>getpeername()</code>	✔ Server	To know client details	<code>accept()</code> creates a new socket with remote endpoint (client IP + port) stored in kernel socket structure ; required for logging, access control, auditing
After <code>connect()</code>	<code>getpeername()</code>	✔ Client	To know server details	After TCP 3-way handshake, kernel sets peer address in socket state ; useful for verification, debugging, multi-server scenarios (DNS resolution ambiguity)
After <code>exec()</code>	<code>getpeername()</code>	✔ Server	Recover lost client info	<code>exec()</code> replaces process memory, so user-space copy of client address is lost, but kernel still maintains peer info in socket descriptor ; <code>getpeername()</code>

IP address	Port	Result
Wildcard	0	Kernel chooses IP address and port
Wildcard	nonzero	Kernel chooses IP address, process specifies port
Local IP address	0	Process specifies IP address, kernel chooses port
Local IP address	nonzero	Process specifies IP address and port

These two functions are required for the following reasons:

- ❖ After connect successfully returns in a TCP client that does not call bind, *getsockname* returns the local IP address and local port number assigned to the connection by the kernel.
- ❖ After calling bind with a port number of 0 (telling the kernel to choose the local portnumber), *getsockname* returns the local port number that was assigned. *getsockname* can be called to obtain the address family of a socket.
- ❖ In a TCP server that binds the wildcard IP address ([intro/daytimetcpsrv.c](#)), once a connection is established with a client (accept returns successfully), the server can call *getsockname* to obtain the local IP address assigned to the connection.
- ❖ The socket descriptor argument to *getsockname* must be that of the connected socket, and not the listening socket.
- ❖ When a server is execed by the process that calls accept, the only way the server can obtain the identity of the client is to call *getpeername*.

The `sockfd_to_family` function shown in the code below returns the address family of a socket.

https://github.com/shichao-an/unpv13e/blob/master/lib/sockfd_to_family.c

```
int
sockfd_to_family(int sockfd)
{
    struct sockaddr_storage ss;
    socklen_t len;

    len = sizeof(ss);
    if (getsockname(sockfd, (SA *) &ss, &len) < 0)
        return(-1);
    return(ss.ss_family);
}
```

The `sockfd_to_family` function shown in the code below returns the address family of a socket.

https://github.com/shichao-an/unpv13e/blob/master/lib/sockfd_to_family.c

This program does the following:

Allocate room for largest socket address structure. Since we do not know what type of socket address structure to allocate, we use a `sockaddr_storage` value, since it can hold any socket address structure supported by the system.

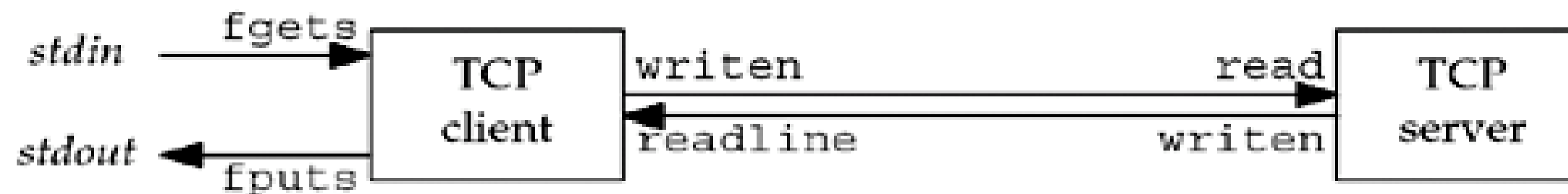
Call `getsockname`. We call `getsockname` and return the address family. The POSIX specification allows a call to `getsockname` on an unbound socket.

Introduction

We will now use the elementary functions from the previous chapter to write a complete TCP client/server example. Our simple example is an echo server that performs the following steps:

1. The client reads a line of text from its standard input and writes the line to the server.
2. The server reads the line from its network input and echoes the line back to the client.
3. The client reads the echoed line and prints it on its standard output.

The figure below depicts this simple client/server:



The echo client/server is a valid, simple example of a network application. To expand this example into your own application, all you need to do is change what the server does with the input it receives from its clients.

Besides running the client/server in normal mode (type in a line and watch it echo), we examine lots of boundary conditions:

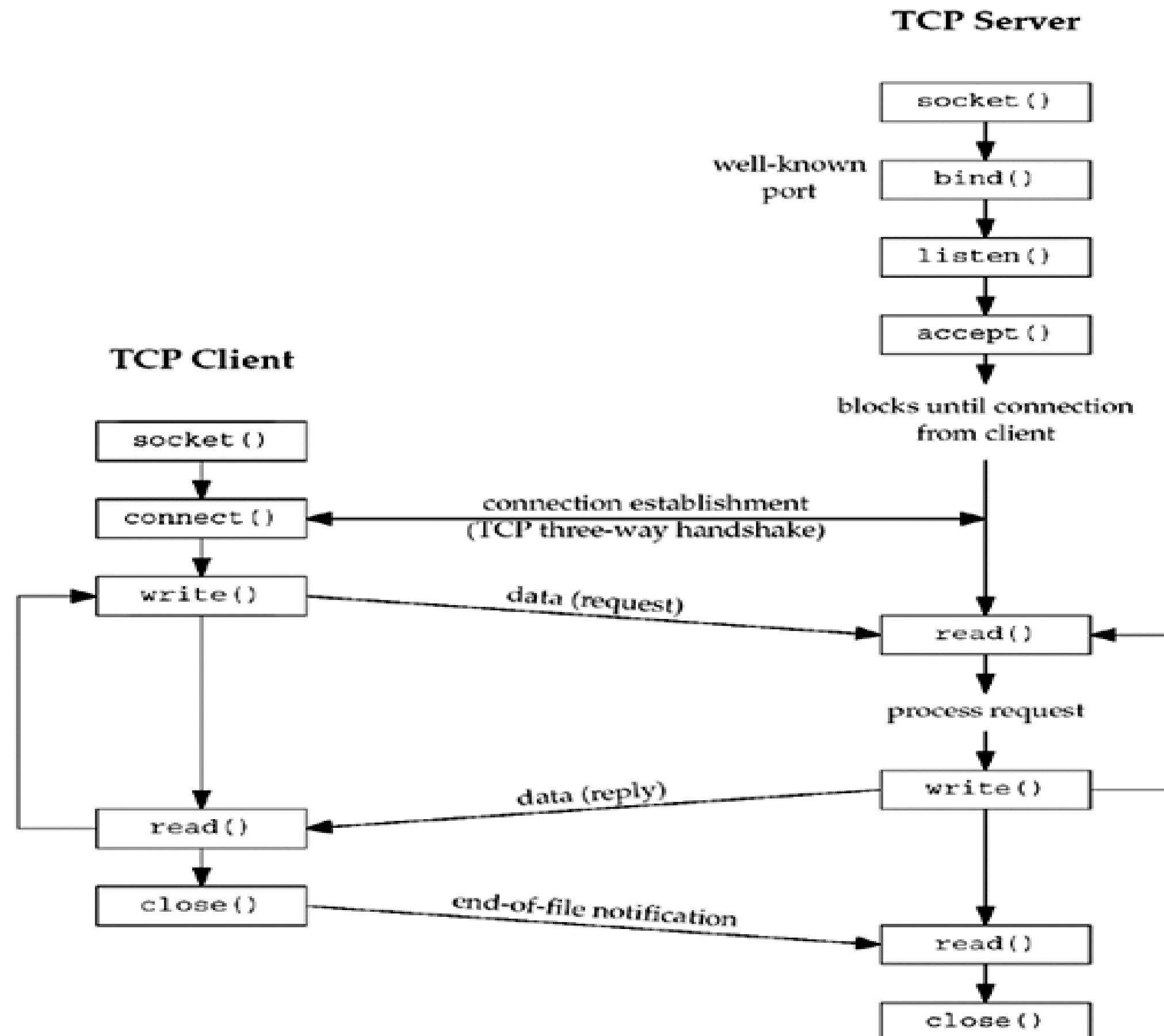
- What happens when the client and server are started?
- What happens when the client terminates normally?
- What happens to the client if the server process terminates before the client is done?
- What happens to the client if the server host crashes?

In all these examples, we have "hard-coded" protocol-specific constants such as addresses and ports.

There are two reasons for this:

- We must understand exactly what needs to be stored in the protocol-specific address structures
- We have not yet covered the library functions that can make this more portable

Our TCP client and server follow the flow of functions that we diagrammed





R #include "unp.h"

E int

```
main(int argc, char **argv)
{
    int listenfd, connfd;
    pid_t childpid;
    socklen_t clilen;
    struct sockaddr_in cliaddr, servaddr;
    listenfd = Socket(AF_INET, SOCK_STREAM, 0);
    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    servaddr.sin_port = htons(SERV_PORT);
    Bind(listenfd, (SA *) &servaddr, sizeof(servaddr));
    Listen(listenfd, LISTENQ);
    for ( ;; ) {
        clilen = sizeof(cliaddr);
        connfd = Accept(listenfd, (SA *) &cliaddr, &clilen);
        if ( (childpid = Fork()) == 0) { /* child process */
            Close(listenfd); /* close listening socket */
            str_echo(connfd); /* process the request */
            exit(0);
        }
        Close(connfd); /* parent closes connected socket */
    }
```

unction

Go, change the world

The below code is the concurrent server program:

The above code does the following:

Create socket, bind server's well-known port

- ❖ A TCP socket is created.
- ❖ An Internet socket address structure is filled in with the wildcard address (INADDR_ANY) and the server's well-known port (SERV_PORT, which is defined as 9877 in our [unp.h](#) header).
- ❖ Binding the wildcard address tells the system that we will accept a connection destined for any local interface, in case the

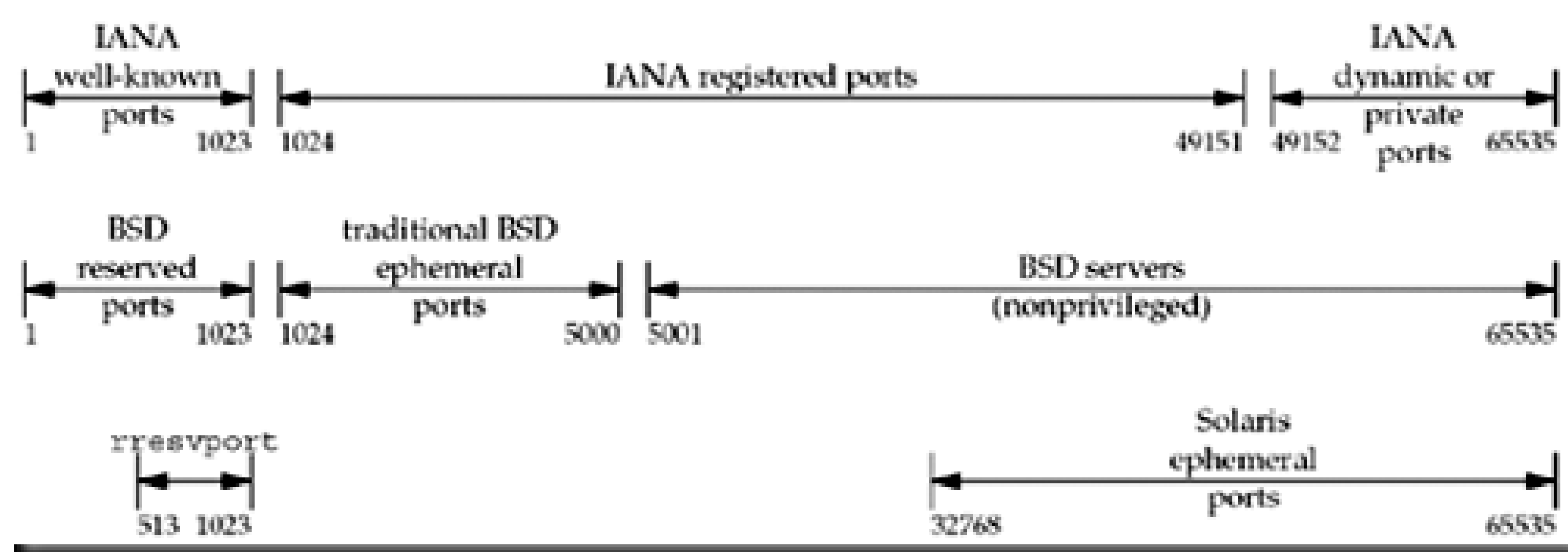


Figure 2.10

Wait for client connection to complete

The server blocks in the call to accept, waiting for a client connection to complete.

Concurrent server

For each client, fork spawns a child, and the child handles the new client. The child closes the listening socket and the parent closes the connected socket. ([Section 4.8](#))


```
#include "unp.h"
void
str_echo(int sockfd)
{
    ssize_t n;
    char buf[MAXLINE];
    again:
    while ( (n = read(sockfd, buf,
        MAXLINE)) > 0)
        Writen(sockfd, buf, n);
    if (n < 0 && errno == EINTR)
        goto again;
    else if (n < 0)
        err_sys("str_echo: read error");
}
```

The function `str_echo` performs the server processing for each client: It reads data from the client and echoes it back to the client.

The above code does the following:

Read a buffer and echo the buffer:

- ❖ `read` reads data from the socket and the line is echoed back to the client by `writen`.
- ❖ If the client closes the connection (the normal scenario), the receipt of the client's FIN causes the child's read to return 0. This causes the `str_echo` function to return, which terminates the child.

```
#include "unp.h"
int
main(int argc, char **argv)
{
    int sockfd;
    struct sockaddr_in servaddr;
    if (argc != 2)
        err_quit("usage: tcpcli <IPaddress>");
    sockfd = Socket(AF_INET, SOCK_STREAM,
0);
    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(SERV_PORT);
    Inet_pton(AF_INET, argv[1],
&servaddr.sin_addr);
    Connect(sockfd, (SA *) &servaddr,
sizeof(servaddr));
    str_cli(stdin, sockfd); /* do it all */
    exit(0);
}
```

The above code does the following:

Create socket, fill in Internet socket address structure

A TCP socket is created and an Internet socket address structure is filled in with the server's IP address and port number. The server's IP address is taken from the command-line argument and the server's well-known port (SERV_PORT) is from our unp.h header.

Connect to server

connect establishes the connection with the server. The function str_cli handles the rest of the client processing.

```
#include "unp.h"
void
str_cli(FILE *fp, int sockfd)
{
    char sendline[MAXLINE], recvline[MAXLINE];

    while (Fgets(sendline, MAXLINE, fp) != NULL)
    {
        Writen(sockfd, sendline, strlen(sendline));

        if (Readline(sockfd, recvline, MAXLINE) == 0)
            err_quit("str_cli: server terminated prematurely");

        Fputs(recvline, stdout);
    }
}
```

The str_cli function handles the client processing loop: It reads a line of text from standard input, writes it to the server, reads back the server's echo of the line, and outputs the echoed line to standard output.

The above code does the following:

Read a line, write to server

fgets reads a line of text and writen sends the line to the server.

Read echoed line from server, write to standard output

readline reads the line echoed back from the server and fputs writes it to standard output.

Return to main

The loop terminates when fgets returns a null pointer, which occurs when it encounters either an end-of-file (EOF) or an error. Our Fgets wrapper function checks for an error and aborts if one occurs, so Fgets returns a null pointer only when an end-of-file is encountered.

Although the TCP example is small, it is essential that we understand:

How the client and server start and end,

What happens when something goes wrong:

- ❖ the client host crashes,
- ❖ the client process crashes,
- ❖ network connectivity is lost

Only by understanding these boundary conditions, and their interaction with the TCP/IP protocols, can we write robust clients and servers that can handle these conditions.

Start the server in the background

First, we start the server in the background:

```
linux % tcpserver01 &  
[1] 17870
```

When the server starts, it calls socket, bind, listen, and accept, blocking in the call to accept.

Run netstat

Before starting the client, we run the netstat program to verify the state of the server's listening socket.

linux % netstat -a

Active Internet connections (servers and established)

<i>Proto</i>	<i>Recv-Q</i>	<i>Send-Q</i>	<i>Local Address</i>	<i>Foreign Address</i>	<i>State</i>
<i>tcp</i>	<i>0</i>	<i>0</i>	<i>*:9877</i>	<i>*:*</i>	<i>LISTEN</i>

- ❖ This command shows the status of all sockets on the system. We must specify the -a flag to see listening sockets.
- ❖ In the output, a socket is in the LISTEN state with a wildcard for the local IP address and a local port of 9877. netstat prints an asterisk for an IP address of 0 (INADDR_ANY, the wildcard) or for a port of 0.

Start the client on the same host

We then start the client on the same host, specifying the server's IP address of 127.0.0.1 (the loopback address). We could have also specified the server's normal (nonloopback) IP address.

linux % tcpcli01 127.0.0.1

The client calls `socket`, and `connect` which causes TCP's three-way handshake. When the three-way handshake completes, `connect` returns in the client and `accept` returns in the server. The connection is established. The following steps then take place:

1. The client calls `str_cli`, which will block in the call to `fgets`.
2. When `accept` returns in the server, it calls `fork` and the child calls `str_echo`. This function calls `readline`, which calls `read`, which blocks while waiting for a line to be sent from the client.
3. The server parent, on the other hand, calls `accept` again, and blocks while waiting for the next client connection.

Notes from the previous three steps:

- ❖ All three processes are asleep (blocked): client, server parent, and server child.
- ❖ We purposely list the client step first, and then the server steps when the three-way handshake completes. This is because `accept` returns one-half of the RTT after `connect` returns.

❑ On the client side, `connect` returns when the second segment of the handshake is received

❑ On the server side, `accept` does not return until the third segment of the handshake is received.

Run netstat after connection completes

Since we are running the client and server on the same host, netstat now shows two additional lines of output, corresponding to the TCP connection:

linux % netstat -a

Active Internet connections (servers and established)

<i>Proto</i>	<i>Recv-Q</i>	<i>Send-Q</i>	<i>Local Address</i>	<i>Foreign Address</i>	<i>State</i>
<i>tcp</i>	<i>0</i>	<i>0</i>	<i>local host:9877</i>	<i>localhost:42758</i>	<i>ESTABLISHED</i>
<i>tcp</i>	<i>0</i>	<i>0</i>	<i>local host:42758</i>	<i>localhost:9877</i>	<i>ESTABLISHED</i>
<i>Tcp</i>	<i>0</i>	<i>0</i>	<i>*:9877</i>	<i>*:*</i>	<i>LISTEN</i>

➤ The first ESTABLISHED line corresponds to the server child's socket, since the local port is 9877.

➤ The second ESTABLISHED lines is the client's socket, since the local port is 42758.

If we were running the client and server on different hosts, the client host would display only the client's socket, and the server host would display only the two server sockets.

Run ps to check process status and relationship

linux % ps -t pts/0 -o pid,ppid,tt,stat,args,wchan

<i>PID</i>	<i>PPID</i>	<i>TT</i>	<i>STAT</i>	<i>COMMAND</i>	<i>WCHAN</i>
<i>22038</i>	<i>22036</i>	<i>pts/6</i>	<i>S</i>	<i>-bash</i>	<i>wait4</i>
<i>17870</i>	<i>22038</i>	<i>pts/6</i>	<i>S</i>	<i>./tcpserv01</i>	<i>wait_for_connect</i>
<i>19315</i>	<i>17870</i>	<i>pts/6</i>	<i>S</i>	<i>./tcpserv01</i>	<i>tcp_data_wait</i>
<i>19314</i>	<i>22038</i>	<i>pts/6</i>	<i>S</i>	<i>./tcpcli01 127.0</i>	<i>read_chan</i>

Very specific arguments to ps are used:

- ❖ The TT column (pts/6): client and server are run from the same window, pseudo-terminal number 6.
- ❖ The PID and PPID columns show the parent and child relationships.
 - ❑ The first tcpserv01 line is the parent and the second tcpserv01 line is the child since the PPID of the child is the parent's PID.
 - ❑ The PPID of the parent is the shell (bash).
- ❖ The STAT column for all three of our network processes is "S", meaning the process is sleeping (waiting for something).
- ❖ The WCHAN column specifies the condition when a process is asleep.
 - ❑ Linux prints wait_for_connect when a process is blocked in either accept or connect, tcp_data_wait when a process is blocked on socket input or output, or read_chan when a process is blocked on terminal I/O.
 - ❑ In [ps\(1\)](#), WCHAN column indicates the name of the kernel function in which the process is sleeping, a "-" if the process is running, or a "*" if the process is multi-threaded and ps is not displaying threads.



At this point, the connection is established and whatever we type to the client is echoed back.

linux % tcpcli01 127.0.0.1

we showed this line earlier

hello, world

we now type this

hello, world

and the line is echoed

good bye

good bye

^D

Control-D is our terminal EOF character

If we immediately execute netstat, we have:

```
linux % netstat -a | grep 9877
```

tcp	0	0	*:9877	*.*	LISTEN
tcp	0	0	localhost:42758	localhost:9877	TIME_WAIT

This time we pipe the output of netstat into grep, printing only the lines with our server's well-known port:

- ❖ The client's side of the connection (since the local port is 42758) enters the TIME_WAIT state
- ❖ The listening server is still waiting for another client connection.

- The following steps are involved in the normal termination of client and server:
- When we type our EOF character, `fgets` returns a null pointer and the function `str_cli` ([Section 5.5](#)) returns.
- `str_cli` returns to the client main function ([Section 5.5](#)), which terminates by calling `exit`.
- Part of process termination is the closing of all open descriptors, so the client socket is closed by the kernel. This sends a FIN to the server, to which the server TCP responds with an ACK. This is the first half of the TCP connection termination sequence. At this point, the server socket is in the `CLOSE_WAIT` state and the client socket is in the `FIN_WAIT_2` state ([Figure 2.4](#) and [Figure 2.5](#))
- When the server TCP receives the FIN, the server child is blocked in a call to `read` ([Section 3.8](#)), and `read` then returns 0. This causes the `str_echo` function to return to the server child main. [Errata] [p128]
- The server child terminates by calling `exit`. ([Section 5.2](#))
- All open descriptors in the server child are closed.
 - The closing of the connected socket by the child causes the final two segments of the TCP connection termination to take place: a FIN from the server to the client, and an ACK from the client.
- Finally, the `SIGCHLD` signal is sent to the parent when the server child terminates.
 - This occurs in this example, but we do not catch the signal in our code, and the default action of the signal is to be ignored. Thus, the child enters the zombie state. We can verify this with the `ps` command



```
linux % ps -t pts/6 -o pid,ppid,tty,stat,args,wchan
```

PID	PPID	TT	STAT	COMMAND	WCHAN
22038	22036	pts/6	S	-bash	read_chan
17870	22038	pts/6	S	./tcpserv01	wait_for_connect
19315	17870	pts/6	Z	[tcpserv01 <defu	do_exit

The STAT of the child is now Z (for zombie).

We need to clean up our zombie processes and doing this requires dealing with Unix signals. The next section will give an overview of signal handling.